

Sistema Integral para Terminal de Transporte Municipal

Erick Albornoz
María José Noreña Rodríguez
Sebastián Sánchez

Universidad Cooperativa de Colombia

Harold A. Bolaños

20 de Mayo del 2025

Cali Colombia

Introducción

El presente proyecto tiene como objetivo el diseño e implementación de un sistema integral para la gestión de una terminal de transporte público, orientado a modernizar y automatizar los procesos operativos y administrativos. Este sistema está concebido para centralizar múltiples funciones clave como la programación de rutas, venta personalizada de tiquetes, seguimiento en tiempo real de buses, gestión de andenes, validación de disponibilidad operativa y generación de informes regulatorios y de auditoría.

Una de las características distintivas de esta solución es su enfoque en la accesibilidad y conectividad. El sistema está diseñado para ser utilizado desde múltiples puntos de acceso: ventanillas de atención, dispositivos móviles, kioscos de auto-consulta y paneles informativos. Además, se integra con tecnologías modernas como sistemas GPS de distintos fabricantes, sensores físicos y plataformas de pago electrónico, permitiendo una experiencia de usuario fluida y una operación altamente eficiente.

Desde el punto de vista de la ingeniería de software, el sistema aplica diversos patrones de diseño orientados a objetos, seleccionados estratégicamente según los retos específicos de cada funcionalidad. Estos patrones no solo permiten una arquitectura más flexible y escalable, sino que también favorecen la mantenibilidad del sistema a largo plazo. Se destacan patrones como Builder, Factory Method, Observer, Composite, Strategy, Adapter, Proxy, State, Memento, entre otros, los cuales han sido aplicados a través de diferentes historias de usuario que representan escenarios reales del entorno operativo.

- Historia de usuario 1, 2, 3 con pantallas de ejecución en consola.

Patrones aplicados: Builder + Factory Method

En esta parte del sistema, implementé el patrón Builder para permitir la construcción flexible de rutas. Como cada ruta puede tener diferentes características (nombre, tipo, número de paradas, kilometraje, etc.), y algunas tienen información opcional como rutas de contingencia o zona extendida, el Builder me permitió armar rutas paso a paso, solo con los datos necesarios.

Luego, para crear automáticamente el tipo correcto de ruta según su categoría (urbana, intermunicipal o turística), usé el patrón Factory Method. Este patrón me ayuda a evitar usar condicionales complejos o instanciar directamente cada clase de ruta. Solo paso los datos al Factory y él se encarga de devolverme la instancia adecuada.

Gracias a esta combinación de patrones, el sistema es más organizado, fácil de mantener y escalable, porque puedo agregar nuevos tipos de rutas sin modificar mucho el código existente.

```
PS C:\Users\majit\OneDrive\Escritorio\Trabajo Final Patrones de Diseño Bien> c::; cd 'c:\Users\majit\OneDrive\
& 'C:\Program Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\majit
9810ffb6140d95f92133847484\redhat.java\jdt_ws\Trabajo Final Patrones de Diseño Bien_fc782c3d\bin' 'SistemaTran
Ruta Urbana: Ruta Central
Número de paradas: 15
Kilometraje: 25.7
Condiciones viales: Buena
Tipo de vehículo: Bus urbano
Ruta de contingencia: Desvío por obras en calle 5
Límite de pasajeros: 50
Zona de cobertura extendida: Sí
-----
Ruta Intermunicipal: Ruta Cauca-Popayán
Tipo: INTERMUNICIPAL
Número de paradas: 8
Kilometraje: 70.5
Condiciones viales: Variable por lluvias
Tipo vehículo: Bus intermunicipal
Ruta contingencia: Camino alternativo vía La Paila
Límite pasajeros: 40
Zona cobertura extendida: No
Ruta Turística: Ruta Panorámica
Número de paradas: 10
Kilometraje: 30.0
Condiciones viales: Excelente
Tipo de vehículo: Microbús turístico
Límite de pasajeros: 30
Zona de cobertura extendida: Sí
-----
```

Ruta Urbana: Ruta Cali Norte
Número de paradas: 12
Kilometraje: 15.3
Condiciones viales: Pavimento en buen estado
Tipo de vehículo: Bus eléctrico
Límite de pasajeros: 50
Zona de cobertura extendida: Sí

Ruta Intermunicipal: Ruta Silvia - Popayán
Tipo: INTERMUNICIPAL
Número de paradas: 5
Kilometraje: 45.7
Condiciones viales: Curvas, zona montañosa
Tipo vehículo: Buseta
Ruta contingencia: Ruta alterna por El Tambo
Límite pasajeros: 30
Zona cobertura extendida: No
Ruta Turística: Ruta Café y Tradición
Número de paradas: 8
Kilometraje: 60.5
Condiciones viales: Asfalto y caminos destapados
Tipo de vehículo: Jeep Willys
Ruta de contingencia: Desvío por Pueblito Viejo
Límite de pasajeros: 0
Zona de cobertura extendida: Sí

Ruta Urbana: Ruta MIO Centro Histórico
Número de paradas: 10
Kilometraje: 9.8
Condiciones viales: Pavimento mixto
Tipo de vehículo: Bus articulado
Límite de pasajeros: 60
Zona de cobertura extendida: No

Ruta Urbana: Ruta MIO Centro Histórico
Número de paradas: 10
Kilometraje: 9.8
Condiciones viales: Pavimento mixto
Tipo de vehículo: Bus articulado
Límite de pasajeros: 60
Zona de cobertura extendida: No

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Ruta contingencia: Ruta alterna por El Tambo
Límite pasajeros: 30
Zona cobertura extendida: No
Ruta Turística: Ruta Café y Tradición
Número de paradas: 8
Kilometraje: 60.5
Condiciones viales: Asfalto y caminos destapados
Tipo de vehículo: Jeep Willys
Ruta de contingencia: Desvío por Pueblito Viejo
Límite de pasajeros: 0
Zona de cobertura extendida: Sí
-----
Ruta Urbana: Ruta MIO Centro Histórico
Número de paradas: 10
Kilometraje: 9.8
Condiciones viales: Pavimento mixto
Tipo de vehículo: Bus articulado
Límite de pasajeros: 60
Zona de cobertura extendida: No
-----
Bus [Bus-101] actualiza datos...
[PanelLlegadas] Ubicación: Garmin: Carrera 45 #12-34 - Estado: Estado no disponible
[App Móvil] ? Garmin: Carrera 45 #12-34 | Estado: Estado no disponible
Bus [Bus-102] actualiza datos...
[PanelLlegadas] Ubicación: Ubicación HuaweiGPS: 3.451, -76.532 - Estado: Estado HuaweiGPS: Conectado
Bus [Bus-103] actualiza datos...
[App Móvil] ? TomTom: Avenida 10 #55-90 | Estado: TomTom: En ruta
> PS C:\Users\majit\OneDrive\Escritorio\Trabajo Final Patrones de Diseño Bien> █
```

- Historia de Usuario 2: Patrones aplicados: Adapter + Observer

Para esta historia, el reto era integrar dispositivos GPS de diferentes marcas, cada uno con su forma distinta de dar la ubicación. Usé el patrón Adapter para convertir la interfaz de cada marca (como Garmin, Huawei o TomTom) a una interfaz común que llamé IGPSBus. Así, sin importar la marca del GPS, todos los buses funcionan igual desde el punto de vista del sistema.

Después, implementé el patrón Observer para que cada bus, cuando actualiza su ubicación, notifique automáticamente a todos los sistemas interesados, como la app móvil, el panel de llegadas o los despachadores. Esto hace que la comunicación sea automática y en tiempo real, mejorando la experiencia para pasajeros y operadores.

Historia de Usuario 3: Panel de llegadas y salidas en tiempo real

Patrones aplicados: Observer + Composite

En esta historia, el objetivo es mostrar en un panel el estado de los buses (salida, en tránsito, retrasado o por llegar) en tiempo real, y que esta información esté organizada jerárquicamente.

Patrón Composite:

Como el sistema tiene una estructura jerárquica donde una terminal contiene varias zonas, cada zona tiene plataformas, y cada plataforma tiene rutas y buses, usé el patrón Composite para representar esta estructura como si fuera un árbol.

Esto permite tratar a todos los elementos (terminal, zona, plataforma, ruta y bus) de manera uniforme, es decir, que se pueden recorrer y gestionar igual, sin importar si son un contenedor (como una zona) o un elemento final (como un bus). Esto facilita mucho el manejo de los paneles y la organización interna del sistema.

Patrón Observer:

Luego, como cada bus puede cambiar de estado (por ejemplo, pasar de "por llegar" a "en tránsito"), usé el patrón Observer para que todos los elementos relacionados sean notificados automáticamente cuando hay un cambio.

Por ejemplo:

Si un bus cambia su estado, no solo se actualiza el estado en ese bus, sino que también lo sabe la plataforma, la zona, la terminal y los paneles o apps que estén mostrando esa información.

De esta forma, no hay que consultar manualmente cada elemento, sino que todos los observadores se actualizan en tiempo real, sin errores ni retrasos.

Composite: permite manejar toda la estructura (terminal → zona → plataforma → ruta → bus) como un árbol, facilitando recorrer y modificar los elementos sin importar su nivel.

Observer: asegura que cuando el estado de un bus cambia, todos los sistemas que muestran esa información se actualicen automáticamente y en tiempo real (paneles, kioscos, app).

```
=====
Nombre: María José
Universidad: Universidad Cooperativa de Colombia
Campus: Cali
Repositorio Git: https://github.com/mariajosenore/Semana-3-Patrones-de-Dise-o-.git
Historia de usuario:
    Este sistema centraliza y automatiza la gestión de una terminal de transporte público:,
    programación de rutas, venta de tiquetes, ubicación de buses, gestión de andenes y
    generación de informes. Está diseñado para ser accesible desde ventanillas, dispositivos
    móviles, kioscos y paneles informativos, e integrarse con GPS, sensores y sistemas de pago electrónico.
    la lógica del sistema.
=====
Identidad codificada en Base64: TWFyw61hIEpvc8Op
Patrón utilizado: Builder, Factory, Adapter, Composite, Observer,
=====
```

- **Aplicacion metodo get identidad para patron y base 64 codificada con el nombre.**
- Pantallas con el Código

```
J SistemaTransporteApp.java > ...
1  // Sistema de Transporte Público
2  import java.util.ArrayList;
3  import java.util.List;
4
5  // Enum para tipo de ruta
6  enum TipoRuta {
7      URBANA,
8      INTERMUNICIPAL,
9      TURISTICA
10 }
11
12 // Builder para rutas
13 class RutaBuilder {
14     String nombre;
15     TipoRuta tipo;
16     int numeroParadas;
17     double kilometraje;
18     String condicionesViales;
19     String tipoVehiculo;
20     String rutaContingencia;
21     int limitePasajeros;
22     boolean zonaCoberturaExtendida;
23
24     public RutaBuilder setNombre(String nombre) { this.nombre = nombre; return this; }
25     public RutaBuilder setTipo(TipoRuta tipo) { this.tipo = tipo; return this; }
26     public RutaBuilder setNumeroParadas(int num) { this.numeroParadas = num; return this; }
27     public RutaBuilder setKilometraje(double km) { this.kilometraje = km; return this; }
28     public RutaBuilder setCondicionesViales(String condiciones) { this.condicionesViales = condiciones; return this; }
29     public RutaBuilder setTipoVehiculo(String tipoVehiculo) { this.tipoVehiculo = tipoVehiculo; return this; }
30     public RutaBuilder setRutaContingencia(String rutaContingencia) { this.rutaContingencia = rutaContingencia; return this; }
31     public RutaBuilder setLimitePasajeros(int limite) { this.limitePasajeros = limite; return this; }
32     public RutaBuilder setZonaCoberturaExtendida(boolean zona) { this.zonaCoberturaExtendida = zona; return this; }
33 }
34
--
```

J SistemaTransporteApp.java > ...

```
38
39 // Clase base Ruta
40 abstract class Ruta {
41     protected String nombre;
42     protected TipoRuta tipo;
43     protected int numeroParadas;
44     protected double kilometraje;
45     protected String condicionesViales;
46     protected String tipoVehiculo;
47     protected String rutaContingencia;
48     protected int limitePasajeros;
49     protected boolean zonaCoberturaExtendida;
50
51     public Ruta(RutaBuilder builder) {
52         this.nombre = builder.nombre;
53         this.tipo = builder.tipo;
54         this.numeroParadas = builder.numeroParadas;
55         this.kilometraje = builder.kilometraje;
56         this.condicionesViales = builder.condicionesViales;
57         this.tipoVehiculo = builder.tipoVehiculo;
58         this.rutaContingencia = builder.rutaContingencia;
59         this.limitePasajeros = builder.limitePasajeros;
60         this.zonaCoberturaExtendida = builder.zonaCoberturaExtendida;
61     }
62
63     public abstract void mostrarDetalles();
64 }
65
66 // Implementaciones para cada tipo de ruta
67
68 class RutaUrbana extends Ruta {
69     public RutaUrbana(RutaBuilder builder) {
70         super(builder);
71     }
72     @Override
73     public void mostrarDetalles() {
74         System.out.println("Ruta Urbana: " + nombre);
```


J SistemaTransporteApp.java > ...

```
68 class RutaUrbana extends Ruta {
73     public void mostrarDetalles() {
74         System.out.println("Ruta Urbana: " + nombre);
75         System.out.println("Número de paradas: " + numeroParadas);
76         System.out.println("Kilometraje: " + kilometraje);
77         System.out.println("Condiciones viales: " + condicionesViales);
78         System.out.println("Tipo vehículo: " + tipoVehiculo);
79         System.out.println("Ruta contingencia: " + (rutaContingencia != null ? rutaContingencia : "No aplica"));
80         System.out.println("Límite pasajeros: " + limitePasajeros);
81         System.out.println("Zona cobertura extendida: " + (zonaCoberturaExtendida ? "Sí" : "No"));
82         System.out.println(x:"-----");
83     }
84 }
85
86 class RutaIntermunicipal extends Ruta {
87     public RutaIntermunicipal(RutaBuilder builder) {
88         super(builder);
89     }
90     @Override
91     public void mostrarDetalles() {
92         System.out.println("Ruta Intermunicipal: " + nombre);
93         System.out.println("Número de paradas: " + numeroParadas);
94         System.out.println("Kilometraje: " + kilometraje);
95         System.out.println("Condiciones viales: " + condicionesViales);
96         System.out.println("Tipo vehículo: " + tipoVehiculo);
97         System.out.println("Ruta contingencia: " + (rutaContingencia != null ? rutaContingencia : "No aplica"));
98         System.out.println("Límite pasajeros: " + limitePasajeros);
99         System.out.println("Zona cobertura extendida: " + (zonaCoberturaExtendida ? "Sí" : "No"));
100        System.out.println(x:"-----");
101    }
102 }
103
104 class RutaTuristica extends Ruta {
105     public RutaTuristica(RutaBuilder builder) {
106         super(builder);
107     }
108 }
```

J SistemaTransporteApp.java > ...

```
86 class RutaIntermunicipal extends Ruta {
91     public void mostrarDetalles() {
98         System.out.println("Límite pasajeros: " + limitePasajeros);
99         System.out.println("Zona cobertura extendida: " + (zonaCoberturaExtendida ? "Sí" : "No"));
100         System.out.println(x:"-----");
101     }
102 }
103
104 class RutaTuristica extends Ruta {
105     public RutaTuristica(RutaBuilder builder) {
106         super(builder);
107     }
108     @Override
109     public void mostrarDetalles() {
110         System.out.println("Ruta Turística: " + nombre);
111         System.out.println("Número de paradas: " + numeroParadas);
112         System.out.println("Kilometraje: " + kilometraje);
113         System.out.println("Condiciones viales: " + condicionesViales);
114         System.out.println("Tipo vehículo: " + tipoVehiculo);
115         System.out.println("Ruta contingencia: " + (rutaContingencia != null ? rutaContingencia : "No aplica"));
116         System.out.println("Límite pasajeros: " + limitePasajeros);
117         System.out.println("Zona cobertura extendida: " + (zonaCoberturaExtendida ? "Sí" : "No"));
118         System.out.println(x:"-----");
119     }
120 }
121
122 // Factory para crear rutas según tipo
123 class RutaFactory {
124     public static Ruta crearRuta(RutaBuilder builder) {
125         switch (builder.tipo) {
126             case URBANA:
127                 return new RutaUrbana(builder);
128             case INTERMUNICIPAL:
129                 return new RutaIntermunicipal(builder);
130             case TURISTICA:
```

J SistemaTransporteApp.java > ...

```
231 class Bus {
232     private String id;
233     private IGPSBus gps;
234     private List<Observador> observadores = new ArrayList<>();
235
236     public Bus(String id, IGPSBus gps) {
237         this.id = id;
238         this.gps = gps;
239     }
240
241     public void agregarObservador(Observador o) {
242         if (!observadores.contains(o)) {
243             observadores.add(o);
244         }
245     }
246
247     public void eliminarObservador(Observador o) {
248         observadores.remove(o);
249     }
250
251     public void notificarObservadores(String mensaje) {
252         for (Observador o : observadores) {
253             o.actualizar(mensaje);
254         }
255     }
256
257     public void actualizarGPS() {
258         String posicion = gps.obtenerCoordenadas();
259         System.out.println("Bus " + id + " GPS actualizado: " + posicion);
260         notificarObservadores("Bus " + id + " está en " + posicion);
261     }
262 }
263
```

J SistemaTransporteApp.java > ...

```
166 class GarminAdapter implements IGPSBus {
174     public String obtenerCoordenadas() {
175         return garmin.obtenerPosicion();
176     }
177 }
178
179 class HuaweiAdapter implements IGPSBus {
180     private HuaweiGPS huawei;
181
182     public HuaweiAdapter(HuaweiGPS huawei) {
183         this.huawei = huawei;
184     }
185
186     @Override
187     public String obtenerCoordenadas() {
188         return huawei.getCoordenadas();
189     }
190 }
191
192 class TomTomAdapter implements IGPSBus {
193     private TomTomGPS tomtom;
194
195     public TomTomAdapter(TomTomGPS tomtom) {
196         this.tomtom = tomtom;
197     }
198
199     @Override
200     public String obtenerCoordenadas() {
201         return tomtom.coordenadasGPS();
202     }
203 }
204
205 // --- Observador ---
206
207 interface Observador {
208     void actualizar(String info);
209 }
```

J SistemaTransporteApp.java > ...

```
266 public class SistemaTransporteApp {
270     public static void main(String[] args) {
276         rutas.add(RutaFactory.crearRuta(new RutaBuilder()
277             .setNombre(nombre:"Ruta Central")
278             .setTipo(TipoRuta.URBANA)
279             .setNumeroParadas(num:15)
280             .setKilometraje(km:25.7)
281             .setCondicionesViales(condiciones:"Buena")
282             .setTipoVehiculo(tipoVehiculo:"Bus urbano")
283             .setRutaContingencia(rutaContingencia:"Desvío por obras en calle 5")
284             .setLimitePasajeros(limite:50)
285             .setZonaCoberturaExtendida(zona:true)));
286
287         rutas.add(RutaFactory.crearRuta(new RutaBuilder()
288             .setNombre(nombre:"Ruta Cauca-Popayán")
289             .setTipo(TipoRuta.INTERMUNICIPAL)
290             .setNumeroParadas(num:8)
291             .setKilometraje(km:70.5)
292             .setCondicionesViales(condiciones:"Variable por lluvias")
293             .setTipoVehiculo(tipoVehiculo:"Bus intermunicipal")
294             .setRutaContingencia(rutaContingencia:"Camino alternativo vía La Paila")
295             .setLimitePasajeros(limite:40)
296             .setZonaCoberturaExtendida(zona:false)));
297
298         rutas.add(RutaFactory.crearRuta(new RutaBuilder()
299             .setNombre(nombre:"Ruta Panorámica")
300             .setTipo(TipoRuta.TURISTICA)
301             .setNumeroParadas(num:10)
302             .setKilometraje(km:30.0)
303             .setCondicionesViales(condiciones:"Excelente")
304             .setTipoVehiculo(tipoVehiculo:"Microbús turístico")
305             .setLimitePasajeros(limite:30)
306             .setZonaCoberturaExtendida(zona:true)));
307
308         // Rutas adicionales
309         rutas.add(RutaFactory.crearRuta(new RutaBuilder()
310             .setNombre(nombre:"Ruta Bogotá-Medellín")
311             .setTipo(TipoRuta.INTERMUNICIPAL)
312             .setNumeroParadas(num:12)
313             .setKilometraje(km:150.0)
314             .setCondicionesViales(condiciones:"Buena")
315             .setTipoVehiculo(tipoVehiculo:"Bus intermunicipal")
316             .setRutaContingencia(rutaContingencia:"Desvío por obras en calle 10")
317             .setLimitePasajeros(limite:40)
318             .setZonaCoberturaExtendida(zona:true)));
319     }
```

J SistemaTransporteApp.java > ...

```
266 public class SistemaTransporteApp {
270     public static void main(String[] args) {
271
272         // Crear GPS y adaptadores
273         GarminGPS garminGPS = new GarminGPS();
274         HuaweiGPS huaweiGPS = new HuaweiGPS();
275         TomTomGPS tomTomGPS = new TomTomGPS();
276
277         IGPSBus garminAdapter = new GarminAdapter(garminGPS);
278         IGPSBus huaweiAdapter = new HuaweiAdapter(huaweiGPS);
279         IGPSBus tomTomAdapter = new TomTomAdapter(tomTomGPS);
280
281         // Crear buses con GPS
282         Bus bus1 =
283             new Bus(id:"Bus 101", garminAdapter);
284         Bus bus2 = new Bus(id:"Bus 202", huaweiAdapter);
285         Bus bus3 = new Bus(id:"Bus 303", tomTomAdapter);
286
287         // Crear observadores
288         Panelllegadas panel = new Panelllegadas();
289         AplicacionMovil appMovil = new AplicacionMovil();
290
291         // Asociar observadores a buses
292         bus1.agregarObservador(panel);
293         bus1.agregarObservador(appMovil);
294
295         bus2.agregarObservador(panel);
296
297         bus3.agregarObservador(appMovil);
298
299         // Actualizar GPS y notificar
300         bus1.actualizarGPS();
301         bus2.actualizarGPS();
302         bus3.actualizarGPS();
303     }
304 }
```

Aquí podra encontrar el resto del Código

https://github.com/erickcolin2005/ucc_pdoo/tree/Proyecto_Final/Limpio/Proyecto_Final

```

10
11 // Entidad Bus
12 class Bus {
13     String id;
14     boolean conductorDisponible;
15     boolean vehiculoEnBuenEstado;
16
17     public Bus(String id, boolean conductorDisponible, boolean vehiculoEnBuenEstado) {
18         this.id = id;
19         this.conductorDisponible = conductorDisponible;
20         this.vehiculoEnBuenEstado = vehiculoEnBuenEstado;
21     }
22 }
23
24 // Handler base
25 abstract class ValidadorAsignacion {
26     protected ValidadorAsignacion siguiente;
27     public void setSiguiente(ValidadorAsignacion sig) {
28         this.siguiente = sig;
29     }
30     public void validar(Bus bus) {
31         if (conducir(bus)) {
32             if (siguiente != null) siguiente.validar(bus);
33             else System.out.println("Bus asignado correctamente");
34         } else {
35             System.out.println("Asignación fallida en " + this.getClass().getSimpleName());
36         }
37     }
38     protected abstract boolean evaluar(Bus bus);
39 }
40
41 class ValidadorConductor extends ValidadorAsignacion {
42     protected boolean evaluar(Bus bus) {
43         return bus.conductorDisponible;
44     }
45 }
46
47 class ValidadorVehiculo extends ValidadorAsignacion {
48     protected boolean evaluar(Bus bus) {
49         return bus.vehiculoEnBuenEstado;
50     }
51 }
52
53 // Strategy
54 interface EstrategiaAsignacion {
55     void asignar(Bus bus);
56 }
57
58 class AsignacionPorPrioridadRuta implements EstrategiaAsignacion {
59     public void asignar(Bus bus) {
60         System.out.println("Asignando por prioridad de ruta al bus: " + bus.id);
61     }
62 }
63
64 class GestorAsignacion {

```

Historia 4: Asignación de Rutas (Chain of Responsibility + Strategy)

Comienzo con la historia de asignación de rutas, que utiliza los patrones **Chain of Responsibility** y **Strategy**. La idea es validar si un bus está en condiciones para ser asignado a una ruta. Para eso usamos una **cadena de validadores**, donde cada uno revisa un aspecto del bus: primero si el conductor está disponible, y luego si el vehículo está en buen estado. Estos validadores (ValidadorConductor y ValidadorVehiculo) extienden de una clase abstracta ValidadorAsignacion, y se enlazan en cadena con setSiguiente. Si el bus pasa todas las validaciones, el sistema imprime que fue asignado correctamente. Si no, se detiene la cadena y se indica en qué validador falló. Luego, para definir cómo se asigna la ruta, usamos el patrón **Strategy**. Creamos una interfaz EstrategiaAsignacion con un método asignar, y en este caso la implementación concreta es AsignacionPorPrioridadRuta, que asigna los buses según prioridad. Finalmente, el GestorAsignacion combina ambas cosas: la cadena de validadores y la estrategia de asignación. En el main, probamos con un bus válido y vemos cómo fluye toda la lógica. Este enfoque hace que sea muy fácil agregar nuevos validadores o cambiar la estrategia de asignación sin modificar el código existente.

```
98
99 interface Tiquete {
100     String descripcion();
101     double calcularPrecio();
102 }
103
104 class TiqueteBasico implements Tiquete {
105     public String descripcion() {
106         return "Tiquete básico";
107     }
108     public double calcularPrecio() {
109         return 5000;
110     }
111 }
112
113 abstract class TiqueteDecorator implements Tiquete {
114     protected Tiquete tiquete;
115     public TiqueteDecorator(Tiquete t) {
116         this.tiquete = t;
117     }
118 }
119
120 class SeguroViaje extends TiqueteDecorator {
121     public SeguroViaje(Tiquete t) {
122         super(t);
123     }
124     public String descripcion() {
125         return tiquete.descripcion() + ", con Seguro de Viaje";
126     }
127     public double calcularPrecio() {
128         return tiquete.calcularPrecio() + 2000;
129     }
130 }
131
132 class DescuentoEstudiante extends TiqueteDecorator {
133     public DescuentoEstudiante(Tiquete t) {
134         super(t);
135     }
136     public String descripcion() {
137         return tiquete.descripcion() + ", con Descuento Estudiante";
138     }
139     public double calcularPrecio() {
140         return tiquete.calcularPrecio() * 0.8;
141     }
142 }
143
144 class ProxyTiquete {
145     private String rolUsuario;
146     public ProxyTiquete(String rolUsuario) {
147         this.rolUsuario = rolUsuario;
148     }
149     public void mostrar() {
150     }
151 }
```

Historia 5: Venta de Tiquetes (Decorator + Proxy)

Pasamos ahora a la historia de la venta de tiquetes. Aquí aplicamos los patrones **Decorator** y **Proxy**. Primero, usamos **Decorator** para construir un tiquete flexible con servicios adicionales. Partimos de un TiqueteBasico con un precio fijo, y le vamos añadiendo decoradores como SeguroViaje y DescuentoEstudiante. Cada decorador modifica la descripción y el precio del tiquete. Esto permite tener una estructura dinámica, donde los usuarios pueden personalizar sus tiquetes sin necesidad de crear muchas subclases. Por ejemplo, si un tiquete tiene ambos: seguro de viaje y descuento por estudiante, el sistema los va encadenando y ajustando el precio total automáticamente. Después, aplicamos el patrón **Proxy** con la clase ProxyTiquete. Esta clase controla el acceso a ciertas operaciones, como reimprimir un tiquete. Solo los usuarios con rol "ADMIN" tienen permiso para hacerlo. Si lo intenta un usuario común, se bloquea la acción con un mensaje de acceso denegado. Esto nos da un control de seguridad sin cambiar la lógica de venta. En el main se ve todo esto en acción: primero se genera un tiquete decorado, luego se realiza la venta y se intenta reimprimir como usuario normal.


```

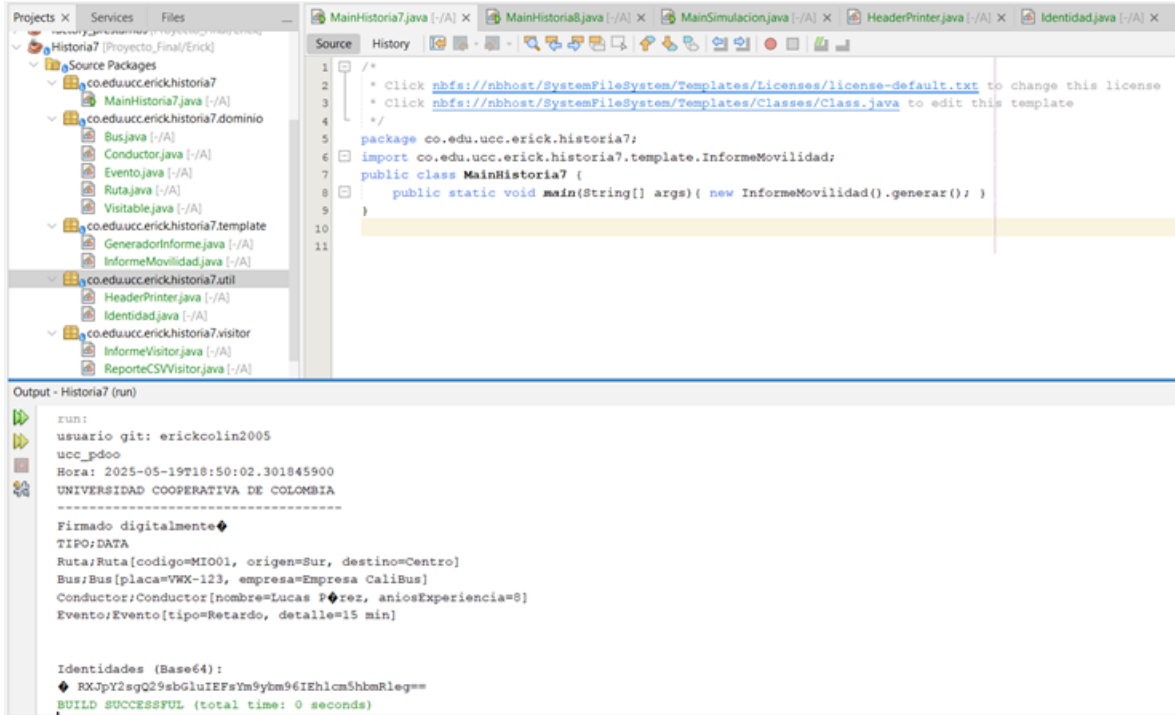
104 interface EstadoBus {
105     void notifyEstado(BusContexto contexto);
106 }
107
108 class EnPlataforma implements EstadoBus {
109     public void notifyEstado(BusContexto contexto) {
110         System.out.println("El bus está en plataforma");
111         contexto.notify("■ Bus en plataforma");
112     }
113 }
114
115 class EnRuta implements EstadoBus {
116     public void notifyEstado(BusContexto contexto) {
117         System.out.println("El bus está en ruta");
118         contexto.notify("■ Bus en ruta");
119     }
120 }
121
122 class Detenido implements EstadoBus {
123     public void notifyEstado(BusContexto contexto) {
124         System.out.println("El bus está detenido");
125         contexto.notify("● Bus detenido");
126     }
127 }
128
129 interface MediatorTerminal {
130     void notifyCambiarEstado(String build, String mensaje);
131 }
132
133 class CentroOperaciones implements MediatorTerminal {
134     public void notifyCambiarEstado(String build, String mensaje) {
135         System.out.println("[Centro de Operaciones] - " + build + " - " + mensaje);
136     }
137 }
138
139 class BusContexto {
140     private String id;
141     private String estado;
142     private MediatorTerminal mediator;
143
144     public BusContexto(String id, MediatorTerminal mediator) {
145         this.id = id;
146         this.mediator = mediator;
147     }
148
149     public void setEstado(Enum estado) {
150         this.estado = estado;
151         estado.notifyEstado(this);
152     }
153
154     public void notify(String mensaje) {
155         mediator.notifyCambiarEstado(id, mensaje);
156     }
157 }

```

Historia 6: Seguimiento de Buses (State + Mediator)

Y por último, para la historia de seguimiento de buses implementé los patrones **State** y **Mediator**. Cada bus puede estar en distintos **estados** durante su operación: en plataforma, en ruta o detenido. Cada uno de estos estados se modela como una clase concreta (EnPlataforma, EnRuta, Detenido) que implementa la interfaz EstadoBus. Cada vez que el estado del bus cambia, se ejecuta la lógica correspondiente y se notifica a un centro de control usando el patrón **Mediator**. El BusContexto mantiene una referencia al estado actual y al MediatorTerminal, que en este caso es el CentroOperaciones. Este mediador se encarga de imprimir una notificación centralizada cuando el estado del bus cambia. Así logramos que los buses no tengan que comunicarse directamente con múltiples partes del sistema, sino que se usa un solo punto de interacción. En el método main, creo un bus, le asigno el mediador y simulo el cambio de estado desde plataforma, luego en ruta y finalmente detenido. Cada cambio lanza una notificación automática, mostrando el flujo completo.

Historia 7



```
1  /*
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
4  */
5  package co.edu.ucc.erick.historia7;
6  import co.edu.ucc.erick.historia7.template.InformeMovilidad;
7  public class MainHistoria7 {
8      public static void main(String[] args) { new InformeMovilidad().generar(); }
9  }
10
11
```

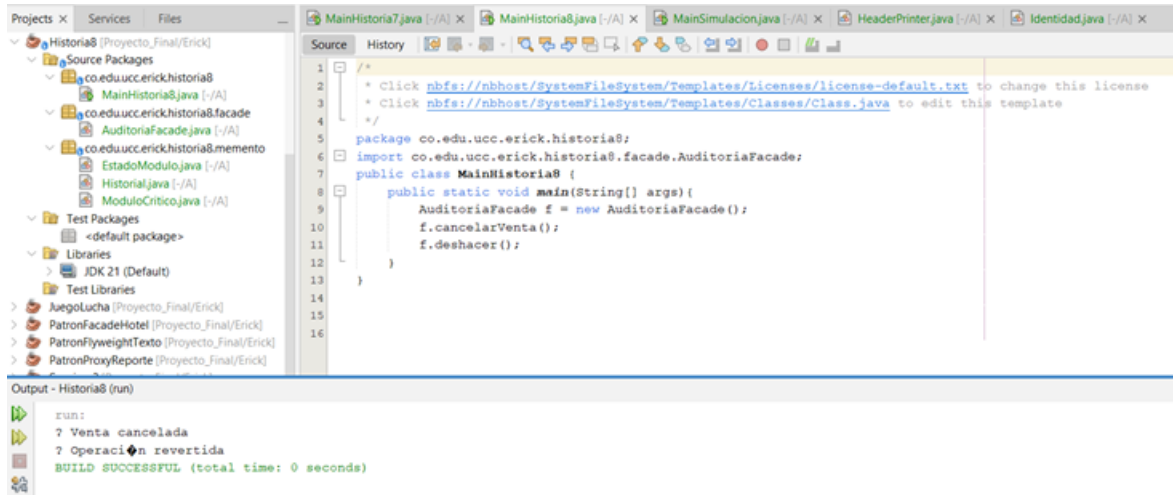
Output - Historia7 (run)

```
run:
usuario git: erickcolin2005
ucc_pdo0
Hora: 2025-05-19T18:50:02.301845900
UNIVERSIDAD COOPERATIVA DE COLOMBIA
-----
Firmado digitalmente
TIPO:DATA
Ruta:Ruta[codigo=MIO01, origen=Sur, destino=Centro]
Bus:Bus[placa=VWX-123, empresa=Empresa CaliBus]
Conductor:Conductor[nombre=Lucas Pórez, añosExperiencia=8]
Evento:Evento[tipo=Retardo, detalle=15 min]

Identities (Base64):
RXpY2sgQ29sbGluIEFsYm9ybm96IEhlcm5hbmRleg==
BUILD SUCCESSFUL (total time: 0 seconds)
```

En la **Historia 7**, desarrollamos una solución orientada a generar informes legales para la **Secretaría de Movilidad**, consolidando datos clave de rutas, buses, conductores y eventos dentro de una terminal. La implementación se basa en dos patrones de diseño estructurales: **Template Method** y **Visitor**. El patrón **Template Method**, aplicado en la clase `GeneradorInforme`, define un flujo estandarizado para construir el documento: recolección de datos, ordenamiento, firma digital simulada y exportación, permitiendo que tipos de informes como `InformeMovilidad` personalicen ciertos pasos sin alterar la estructura general. Por su parte, el patrón **Visitor** permite recorrer de forma flexible las entidades (`Ruta`, `Bus`, `Conductor`, `Evento`) a través del método `generarReporte()` sin modificar sus clases internas, facilitando la extensión del sistema ante futuros requerimientos normativos.

Historia 8



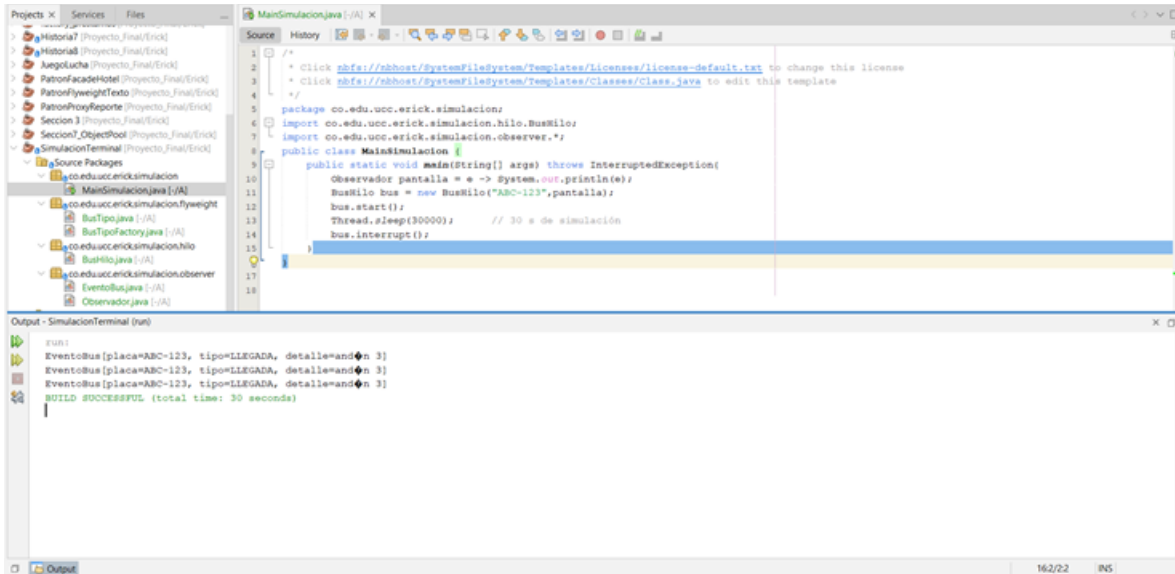
```
1  /*
2  3  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
4  5  */
6  package co.edu.ucc.erick.historia8;
7  import co.edu.ucc.erick.historia8.facade.AuditoriaFacade;
8  public class MainHistoria8 {
9      public static void main(String[] args) {
10         AuditoriaFacade f = new AuditoriaFacade();
11         f.cancelarVenta();
12         f.deshacer();
13     }
14 }
15
16
```

Output - Historia8 (run)

```
run:
? Venta cancelada
? Operaci3n revertida
BUILD SUCCESSFUL (total time: 0 seconds)
```

En la **Historia 8**, se desarrolló un sistema de auditoría para gestionar eventos sensibles en una aplicación crítica, como la cancelación de ventas o la reversión de operaciones. Para ello, se implementaron dos patrones de diseño: **Memento** y **Facade**. El patrón **Memento** permite capturar y almacenar el estado interno de los módulos críticos antes de ejecutar acciones delicadas, como `cancelarVenta()`, posibilitando así una reversión segura mediante el método `deshacer()`, tal como se observa en la ejecución del sistema. Por su parte, el patrón **Facade** se aplicó en la clase `AuditoriaFacade`, que actúa como una interfaz simplificada desde la cual los auditores pueden ejecutar las operaciones de auditoría sin necesidad de conocer los detalles internos de los módulos involucrados (`EstadoModulo`, `ModuloCritico`, `Historial`). Al ejecutar `MainHistoria8`, se evidencia la funcionalidad esperada: una venta es cancelada y la operación es revertida exitosamente, reflejando el correcto uso del patrón **Memento** para restaurar el estado anterior y del patrón **Facade** para centralizar la lógica de auditoría. Esta solución garantiza trazabilidad, encapsulamiento y facilidad de mantenimiento ante futuras auditorías o normativas.

Historia Especial



```
1  /*  
2  * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/default.txt to change this license  
3  * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template  
4  */  
5  package co.edu.ucc.erick.simulacion;  
6  import co.edu.ucc.erick.simulacion.hilo.BusHilo;  
7  import co.edu.ucc.erick.simulacion.observer.*;  
8  public class MainSimulacion {  
9  
10     public static void main(String[] args) throws InterruptedException {  
11         Observador pantalla = e -> System.out.println(e);  
12         BusHilo bus = new BusHilo("ABC-123", pantalla);  
13         bus.start();  
14         Thread.sleep(30000); // 30 s de simulación  
15         bus.interrupt();  
16     }  
17  
18 }
```

Output - SimulationTerminal (run)

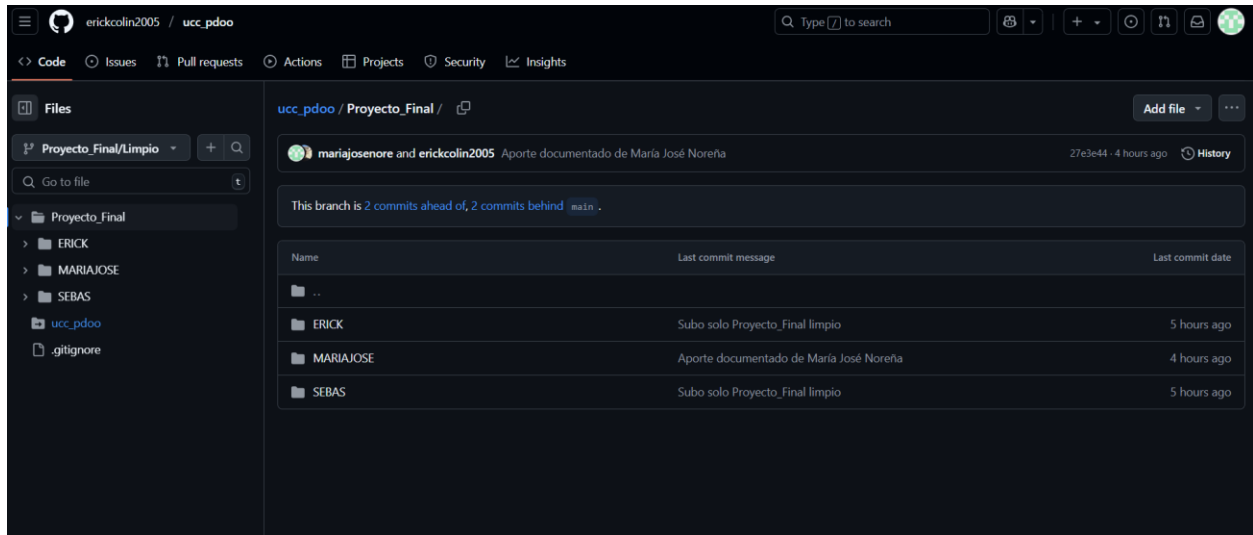
```
EventBus{placa=ABC-123, tipo=LEGADA, detalle=and\n 3}  
EventBus{placa=ABC-123, tipo=LEGADA, detalle=and\n 3}  
EventBus{placa=ABC-123, tipo=LEGADA, detalle=and\n 3}  
BUILD SUCCESSFUL (total time: 30 seconds)
```

En esta **Historia especial** se desarrolló una simulación realista de operación en una terminal de transporte, donde se modela el comportamiento dinámico de un bus mediante **hilos (Thread)** y se integran los patrones **Observer** y **Flyweight**. En la clase `MainSimulacion`, se crea un hilo (`BusHilo`) que representa un bus con placa ABC-123, el cual emite eventos periódicos (como llegada) durante un intervalo de 30 segundos. El patrón **Observer** se implementa para que dichos eventos sean notificados a un observador (pantalla), simulando el monitoreo en tiempo real por parte de sistemas como pantallas de información, kioscos o seguridad. Además, se aplica el patrón **Flyweight** para compartir atributos comunes entre buses (como marca y capacidad), optimizando el uso de memoria al evitar instancias duplicadas. Esta ejecución demuestra cómo, con solo una instancia del bus y un observador, se logran múltiples emisiones de eventos (`EventoBus`), permitiendo evaluar la estabilidad del sistema bajo condiciones de carga controlada, esencial para la validación del monitoreo y la operación en tiempo real de terminales inteligentes.

Link: git hub.

https://github.com/erickcolin2005/ucc_pdoo/tree/Proyecto_Final/Limpio

El cual se trabajo colaborativamente desde las ramas y haciendo los cambios desde cada git como colaboradores, ahi se evidencia un trabajo final limpio e integrado por paquetes y encabezados.



erickcolin2005 / ucc_pdoo

Code Issues Pull requests Actions Projects Security Insights

Files

Proyecto_Final/Limpio

Go to file

Proyecto_Final

- ERICK
- MARIAJOSE
- SEBAS
- ucc_pdoo
- .gitignore

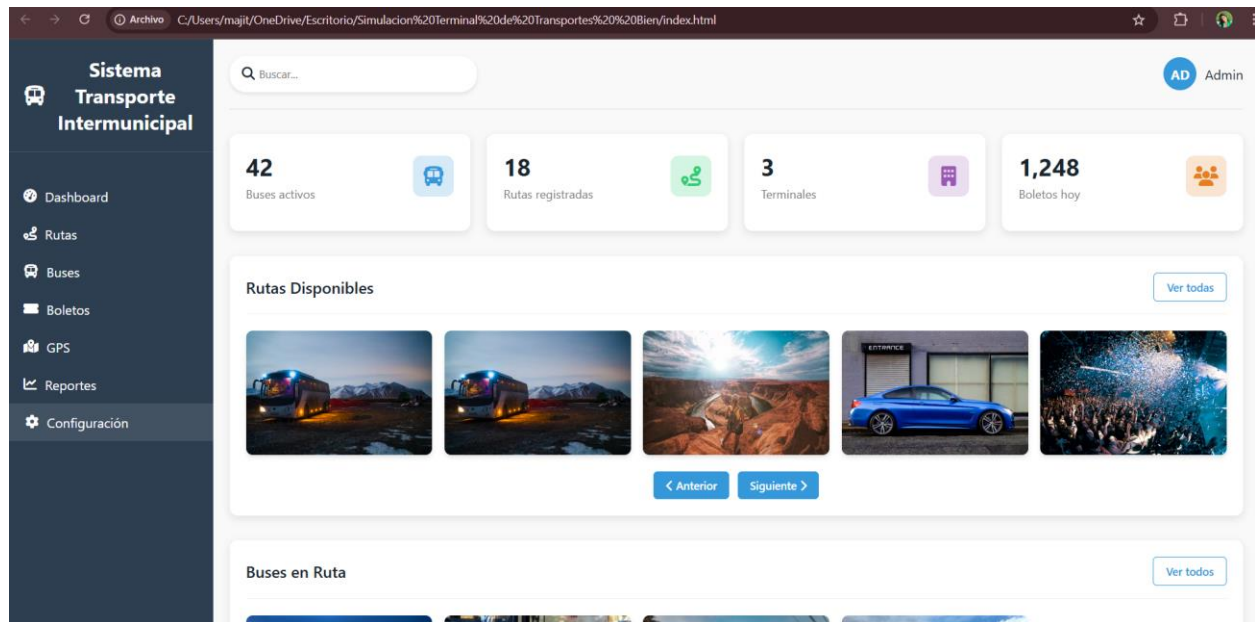
ucc_pdoo / Proyecto_Final

mariajosenore and erickcolin2005 Aporte documentado de María José Noreña 27e3e44 · 4 hours ago History

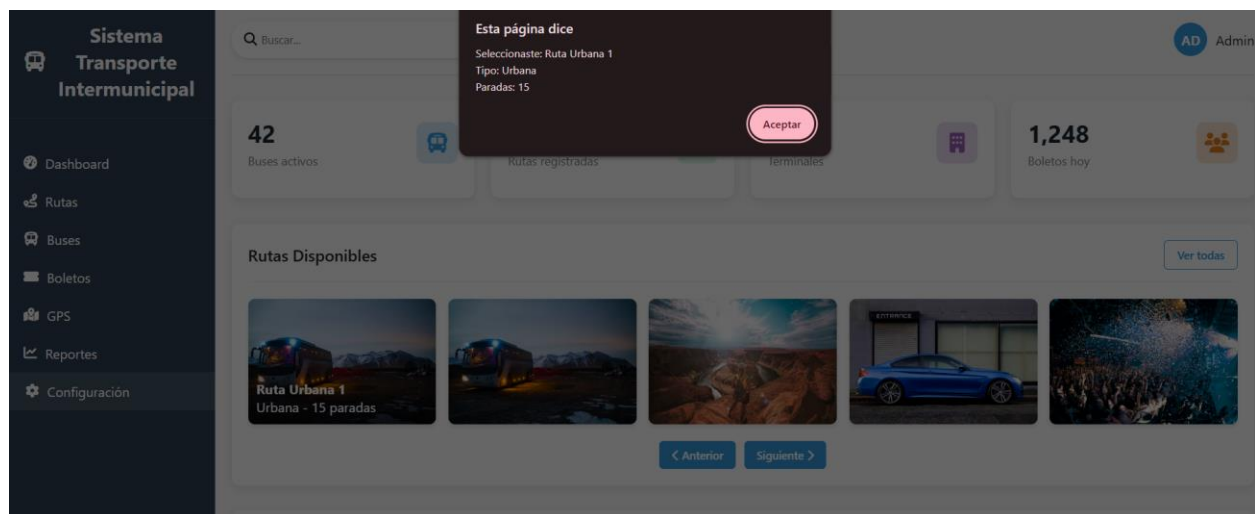
This branch is 2 commits ahead of, 2 commits behind main .

Name	Last commit message	Last commit date
..		
ERICK	Subo solo Proyecto_Final limpio	5 hours ago
MARIAJOSE	Aporte documentado de María José Noreña	4 hours ago
SEBAS	Subo solo Proyecto_Final limpio	5 hours ago

Se creo una página en html simulando un sistema de transportes intermunicipal integrando algunas funciones básicas pero con un alcance a futuro mas integrado con un backend y pues también que quede con responsive a una app.



El cual muestra los buses activos, rutas registradas, las terminales y los boletos comprados, las rutas disponibles y los buses en ruta en la parte izquierda encontramos un dashboard con diferentes opciones.



Sistema Transporte Intermunicipal

- Dashboard
- Rutas
- Buses
- Boletos
- GPS
- Reportes
- Configuración

Esta página dice

Mostrando siguientes rutas...

Aceptar

42

Buses activos



18

Rutas registradas



3

Terminales



1,248

Boletos hoy



Rutas Disponibles



< Anterior

Siguiente >

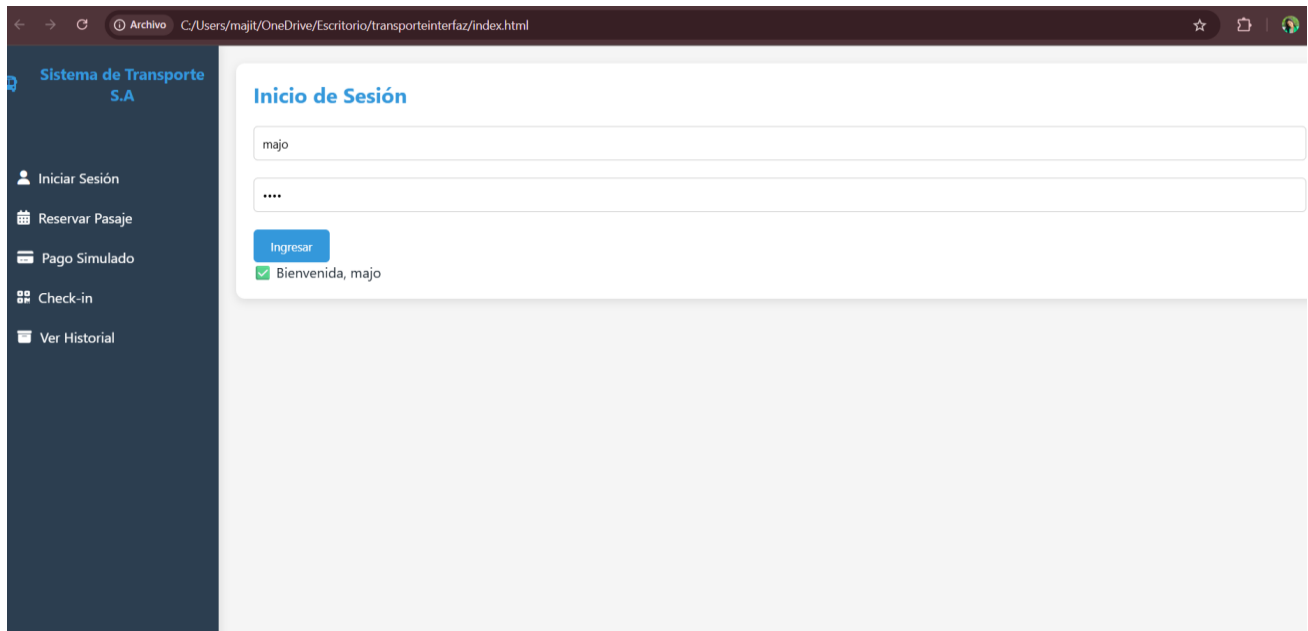
Buses en Ruta



Y aqui encontramos para iniciar sesion y comprar tus tiquetes en linea el cual tiene un inicio de sesion, reservar pasaje, pago simulado, check in y un historial de reserva y pago del tiquete ademas de eso un codigo qr que va a la empresa de transporte y ya este seria nuestro sistema de transportes.

[https://github.com/mariajosenore/Semana-3-Patrones-de-Dise-o-
/tree/main/transporteinterfaz](https://github.com/mariajosenore/Semana-3-Patrones-de-Dise-o-/tree/main/transporteinterfaz)

[https://github.com/mariajosenore/Semana-3-Patrones-de-Dise-o-
/tree/main/Simulacion%20Terminal%20de%20Transportes%20%20Bien](https://github.com/mariajosenore/Semana-3-Patrones-de-Dise-o-/tree/main/Simulacion%20Terminal%20de%20Transportes%20%20Bien)



Sistema de Transporte S.A.

- Iniciar Sesión
- Reservar Pasaje
- Pago Simulado
- Check-in
- Ver Historial

Inicio de Sesión

majo

....

Ingresar

✓ Bienvenida, majo

 Sistema de Transporte S.A

Iniciar Sesión

Reservar Pasaje

Pago Simulado

Check-in

Ver Historial

Reserva de Pasaje

Maria Jose

Cali

20/05/2025

mayo de 2025

↑

↓

DO	LU	MA	MI	JU	VI	SA
27	28	29	30	1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31
1	2	3	4	5	6	7

Borrar

Hoy

 Sistema de Transporte S.A

Iniciar Sesión

Reservar Pasaje

Pago Simulado

Check-in

Ver Historial

Reserva de Pasaje

Maria Jose

Cali

20/05/2025

Reservar

✓

 Reserva realizada con éxito

 Sistema de Transporte
S.A

-  Iniciar Sesión
-  Reservar Pasaje
-  Pago Simulado
-  Check-in
-  Ver Historial

Pago Simulado

Nequi

Nequi

Daviplata

Tarjeta

 Sistema de Transporte
S.A

-  Iniciar Sesión
-  Reservar Pasaje
-  Pago Simulado
-  Check-in
-  Ver Historial

Pago Simulado

Nequi

Pagar

 Pago exitoso con Nequi



Sistema de Transporte S.A



Iniciar Sesión



Reservar Pasaje



Pago Simulado



Check-in



Ver Historial

Check-in con Código QR

Generar QR





Sistema de Transporte S.A



Iniciar Sesión



Reservar Pasaje



Pago Simulado



Check-in



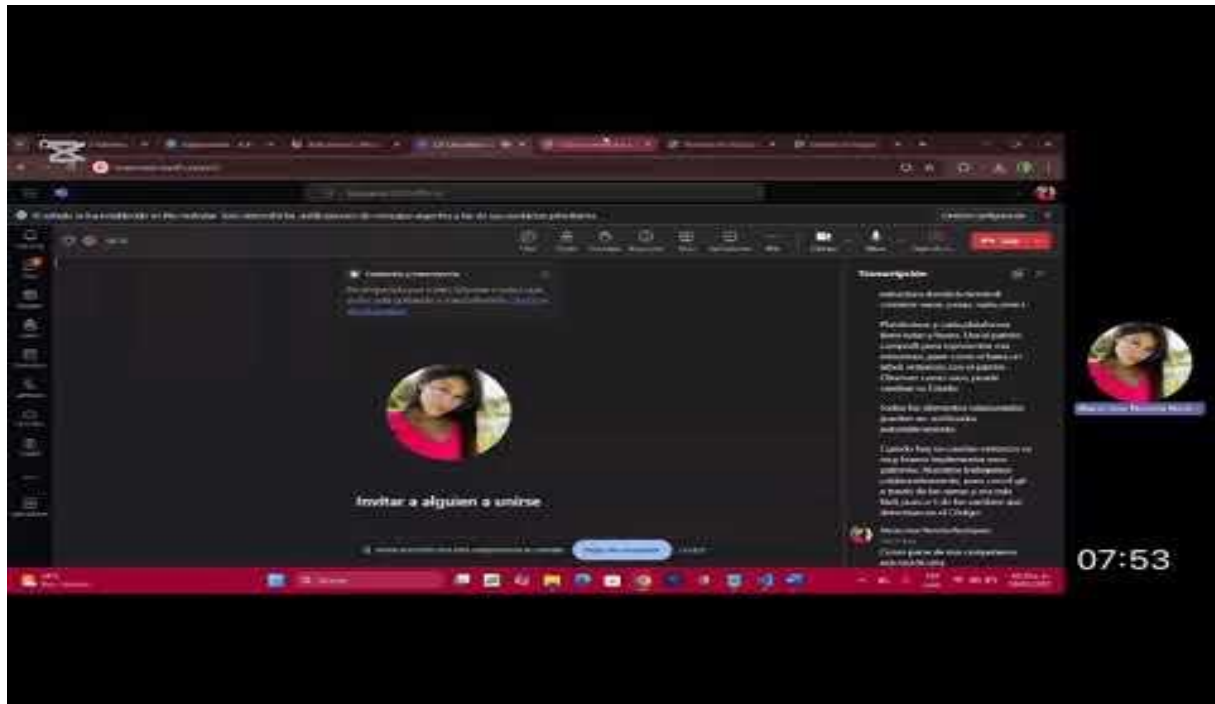
Ver Historial

Historial de Reservas

Pasajero: Maria Jose

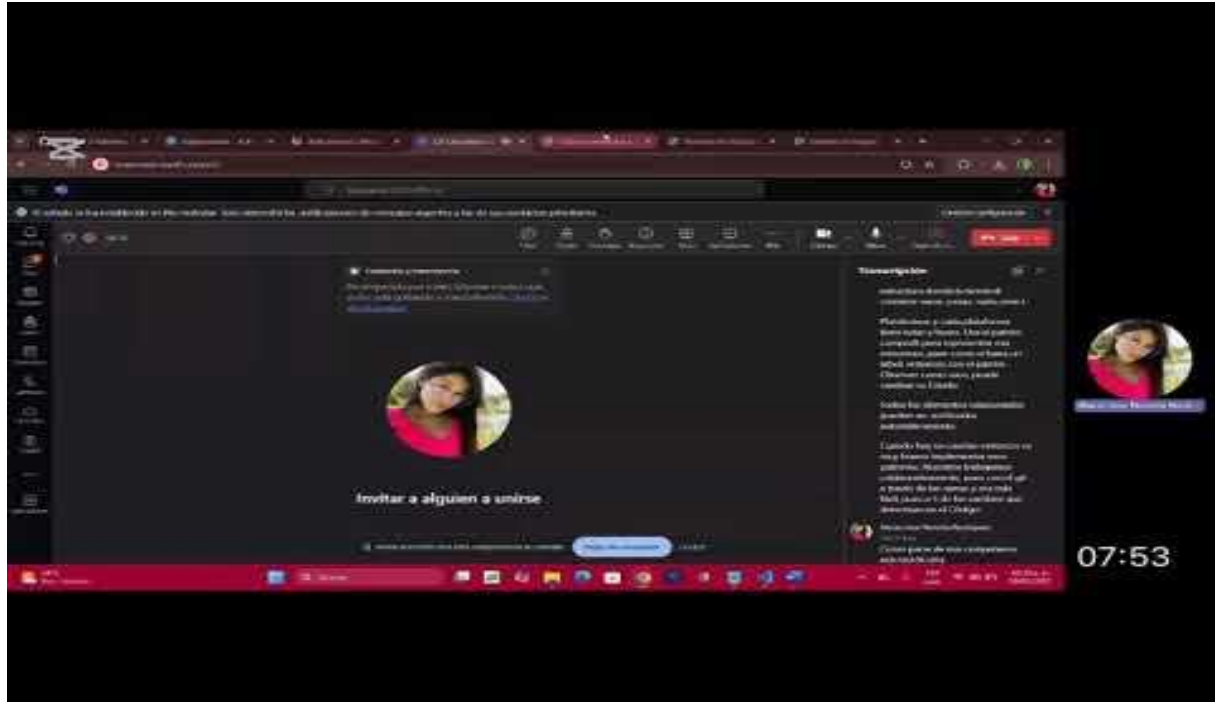
Destino: Cali

Fecha: 2025-05-20



Video Explicacion.

<https://youtu.be/P7Xzc0jWwwI>



Muchas Gracias.